Final Assessment: Part 1 Assessment and Final Project

The kit's two summative assessments on one page. Part 1 ends with the Part 1 assessment in Session 7 (the course's final assessment when Part 1 runs on its own). Part 2 ends with the final project: Lesson 8.2's Maple Street Tool Library, briefed in Session 12, built in the studios of Sessions 13 and 14 and submitted one week later. Rubrics, scoring guidance and instructor answer notes whose numbers come from real runs on MySQL 9.7. For instructors only.

Part 1: **Session 7** | Final project: **Sessions 12–14** | Project rubric **100** points | Portfolio-ready: **80+** | Instructor only

ON THIS PAGE

- **Part 1 Assessment (Session 7)**
- **Final Project: The Maple Street Tool Library** (Sessions 12 to 14): the brief, where the minutes go, the rubric, scoring guidance, the grading run, the scoring sheet, submission, late work, recording course results
- **Answer Notes · Final Project Reference Numbers**: every milestone's reference output, measured

PART 1 ASSESSMENT: THE RIVERSIDE COOKING SCHOOL

The assessment that closes Part 1 (Sessions 1 to 7), run in the second half of Session 7. Each learner works alone for 60 minutes, open book, in one SQL file on a database they have never seen: a fictional cooking school with instructors, classes and students. They build and protect the table that links students to classes, change data safely, and answer eight questions with filters, aggregates, subqueries and joins. Score it with the five-row rubric below. The instructor answer notes hold the reference output and what the common wrong answers print: this section is for instructors only.

Covers **Modules 1–4** | **Session 7**, second half | **60 min** in the room | Open book, alone | Met: **14/20**, no row at 1 | Instructor only

PART 1 ASSESSMENT: THE BRIEF (AS THE INSTRUCTOR SEES IT)

Files. Learners get `labs/module-07/part1-assessment/part1_assessment_starter.sql`: the brief is its header and TODO comments, and running it builds `kit_riverside` (instructors 5 rows, classes 8, students 8), shows placeholder lines and drops the database at the end. Your reference is `labs/module-07/part1-assessment/part1_assessment_solution.sql` (never handed out before scoring). Both pass `python3 labs/check_labs.py module-07`. Programs that skip Session 7's assessment slot can run the same file as a take-home exercise (open book, 48 hours).

TASK	WHAT THE LEARNER DOES	LESSONS
A1	Create `enrollments`: composite primary key, two named foreign keys with chosen rules and a one-sentence reason each, a CHECK for rating 1 to 5, enrolled_on required	2.1, 2.2, 4.1, 4.2
A2	Turn on the given INSERT (15 enrollments, 12 rated) and check the counts	2.3
A3a	Record the errors for two given statements run at the prompt: student 9 (1452) and rating 6 (3819)	2.2, 4.2
A3b	Raise every Italian class by 5.00 with one UPDATE	2.3, 3.1
A3c	Delete the cancelled class 8, then list the classes	2.3, 4.2
B1	March 2026 classes under $70, cheapest first	3.1, 3.2
B2	Per cuisine: class count and average price, highest first, ties by cuisine	3.3
B3	Students with more than two enrollments, with names	3.3, 4.3 (or 3.4)
B4	Per class: enrollments, ratings given, average rating to one decimal (blanks do not count)	3.3, 4.3
B5	Classes above the average price, no typed number	3.4
B6	April 2026 enrollments with student, class and instructor	4.3 (or 3.4)
B7	Students who never enrolled	4.3 anti-join (or 3.4 NOT EXISTS)
B8	Revenue per instructor, every instructor included	4.3

- **Deliverable:** one file, `part1_assessment_<name>.sql`, that runs from start to finish with SOURCE and prints the answers (the starter's structure, every answer under its TODO; A3a's two errors written as comments).
- **Timing:** 10 minutes of setup at 1:25 (rules, data, one run of the starter), 60 minutes of work (Part A about 20, Part B about 35, a final run 5), handed in at 2:35. Learners with agreed extra time get up to 30 more minutes, or finish the file at home within 48 hours.

- **Rules:** open book (lesson pages, notes, workbook, own scripts); no other people, no AI assistants. Joins and subqueries are both accepted wherever both work; Part B is scored on the data AFTER Part A (a skipped Part A step changes later numbers; score each answer against what the learner's own file produced, see the notes).
- **Assessed:** the submitted file against the five rubric rows. The Part 1 review that follows in the room is not scored.

PART 1 ASSESSMENT: THE RUBRIC

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Table design and constraints (A1) · weight 20%	No table, or a table without a primary key; keys missing.	A table that loads the data, but with a single-column key (duplicates possible), keys unnamed or missing one side, or no CHECK; no reasons for the rules.	Composite PRIMARY KEY (student_id, class_id); two named foreign keys referencing students(id) and classes(id); `enrolled_on DATE NOT NULL`; a CHECK that refuses 6; a delete rule per key with a one-sentence reason that fits it.	Also matches column types exactly to the parents (INT), names every constraint, and the reasons weigh history against clean-up (for example RESTRICT for students, CASCADE for classes) or argue a different choice convincingly.
Data changes and integrity (A2, A3) · weight 15%	UPDATE or DELETE without WHERE, or data loaded with checks switched off.	The changes are made but A3a's errors are missing or misread (1451 for a missing student), or a change hits more rows than asked.	A2 loads 15 rows (12 rated); A3a records 1452 (no such student) and 3819 (CHECK), each with what it means; one UPDATE with `WHERE cuisine = 'Italian'`; one DELETE by id; the class list shows 7 classes, Pasta 70.00 and Risotto 60.00.	Also checks the effect of each change with a SELECT and explains why deleting class 8 is safe (no enrollments; or what CASCADE would do if it had some).

CRITERION	1 · BEGINNING	2 · DEVELOPING	3 · PROFICIENT	4 · EXEMPLARY
Filtering, sorting and aggregates (B1 to B4) · weight 25%	Two or more of B1 to B4 missing or not running.	Answers run but one or two are wrong: a missing date bound, `AVG(COALESCE(rating, 0))`, COUNT(*) for ratings, a missing tie-breaker, an aggregate in WHERE.	All four right on the data after Part A: B1 Sourdough 45.00 and Tacos 50.00; B2 Italian and Japanese 65.00, Baking 60.00, Mexican 50.00; B3 Ana, Chloe, Emma with 3; B4 seven classes with blank ratings skipped (Pasta 3 / 2 / 4.5). Each has an ORDER BY.	Also readable: aliases for every computed column, ROUND where a person reads the number, HAVING only for group conditions.
Subqueries and joins (B5 to B8) · weight 25%	Two or more of B5 to B8 missing or not running.	Answers run but one or two are wrong: a typed average in B5, an inner join or IS NULL after an inner join in B7, Nadia missing or a fan-out in B8.	All four right: B5 French Pastry, Fresh Pasta by Hand, Sushi Basics; B6 five April enrollments with three names each; B7 Gina Russo and Hugo Weber; B8 five instructors from Marco Rossi 330.00 to Nadia Haddad 0.00. Joins or subqueries, both accepted.	Also: qualified columns throughout, the anti-join or NOT EXISTS chosen deliberately, and B8's sum robust to a class with no enrollments (CASE or a count of enrollments times price).
Script quality (the whole file) · weight 15%	The file does not run, or answers are not in the file.	Runs only after edits, or stops partway; typed numbers where the database could compute them; no comments.	Runs start to finish with SOURCE and no error; every answer under its TODO; ORDER BY whenever several rows come back; a comment where a choice was made.	Also consistent formatting and aliases a reader can follow without the brief.

SCORING AND THE RESULT

- Score each row 1 to 4 (whole numbers). Report the total out of 20 and, if your program uses percentages, the weighted score (weights in the rubric: 20 / 15 / 25 / 25 / 15).
- **Part 1 met:** a total of 14 or more and no row at 1. That is Proficient on most rows, with room for one Developing row.
- **Met after revision:** a total of 11 to 13, or one row at 1 with a total of 14 or more. The learner fixes the weakest row's tasks in the same file within one week; re-score those rows only.
- **Not yet:** a total of 10 or less, or two rows at 1. Return written feedback (one strength per row, the single change that would help most) and offer the take-home variant once, within two weeks.
- **Run before you read:** SOURCE the file first. A file that stops with an error is scored on what it printed and on the code you can read; the Script quality row takes the hit, not the content rows twice.

PART 1 ASSESSMENT: SCORING GUIDANCE BY ROW

ROW	WHAT TO LOOK FOR
Table design and constraints	`PRIMARY KEY (student_id, class_id)` (a separate id column with no UNIQUE on the pair caps the row at 2: the "only once" rule is not enforced); two `CONSTRAINT ... FOREIGN KEY` lines with ON DELETE rules; `CHECK (rating BETWEEN 1 AND 5)` (or `>= 1 AND <= 5`); the two reason sentences. Any rule pair with a sensible reason is Proficient; CASCADE on students with no reason is Developing.
Data changes and integrity	A3a's two comments name 1452 and 3819 and say what each means; the UPDATE has a WHERE on cuisine (or on ids 1 and 2); the DELETE removes id 8 only. Look for `SET FOREIGN_KEY_CHECKS = 0` (caps the row at 1).
Filtering, sorting and aggregates	Compare B1 to B4 with the reference output below. B1 needs both date bounds (BETWEEN or >= and <) and `price < 70`; B4's average must skip blanks (`AVG(rating)`), and ratings given must be `COUNT(rating)`.
Subqueries and joins	B5 with a subquery (a typed 61.43 prints the right rows today: Developing, see the traps); B7 with LEFT JOIN ... IS NULL, NOT EXISTS or NOT IN (safe here: student ids are never NULL); B8 must show Nadia Haddad with 0.
Script quality	SOURCE runs end to end; ORDER BY on every multi-row answer (B3 by last name, B6 by date, B8 by revenue then name); readable aliases; comments at the choices.

PART 1 ASSESSMENT: SCORING SHEET

Learner: _____ · Date: _____ · Extra time: yes / no · File runs end to end: yes / no

ROW	WEIGHT	SCORE 1–4	EVIDENCE (TASK AND LINE)
Table design and constraints	20%		
Data changes and integrity	15%		
Filtering, sorting and aggregates	25%		
Subqueries and joins	25%		
Script quality	15%		
Total	100%	/ 20	Result: met / met after revision / not yet

PART 1 ASSESSMENT: INSTRUCTOR ANSWER NOTES

A SOUND SOLUTION: STEPS

- A1: a composite primary key, two named foreign keys (the reference: students RESTRICT, a paid booking is history; classes CASCADE, a cancelled class's bookings mean nothing without it), a CHECK, `enrolled_on DATE NOT NULL`, `rating TINYINT` nullable.
- A2: turn on the INSERT; A3a: run the two statements at the prompt and copy the errors into comments; A3b: `UPDATE classes SET price = price + 5.00 WHERE cuisine = 'Italian';`; A3c: `DELETE FROM classes WHERE id = 8;`.
- B1 to B4: WHERE, ORDER BY, GROUP BY, HAVING, COUNT(column) versus COUNT(*). B5: a scalar subquery. B6: four tables joined on their keys. B7: the anti-join. B8: LEFT JOIN from instructors to classes to enrollments, COUNT of an enrollment column, a sum that adds a price only for real enrollments.

KEY NUMBERS (THE REFERENCE RUN ON MYSQL 9.7.2)

The reference solution prints, in order: A2's counts, the classes after A3, then B1 to B8.

```
Database changed
+-------------+-------+
| enrollments | rated |
+-------------+-------+
|          15 |    12 |
+-------------+-------+
+----+---------------------+-------+
| id | title               | price |
+----+---------------------+-------+
|  1 | Fresh Pasta by Hand | 70.00 |
|  2 | Risotto Nights      | 60.00 |
|  3 | Sushi Basics        | 70.00 |
|  4 | Ramen from Scratch  | 60.00 |
|  5 | Sourdough Starter   | 45.00 |
|  6 | French Pastry       | 75.00 |
|  7 | Tacos and Salsas    | 50.00 |
+----+---------------------+-------+
+-------------------+------------+-------+
| title             | class_date | price |
+-------------------+------------+-------+
| Sourdough Starter | 2026-03-21 | 45.00 |
| Tacos and Salsas  | 2026-03-28 | 50.00 |
+-------------------+------------+-------+
+----------+---------+-----------+
| cuisine  | classes | avg_price |
+----------+---------+-----------+
| Italian  |       2 |     65.00 |
| Japanese |       2 |     65.00 |
| Baking   |       2 |     60.00 |
| Mexican  |       1 |     50.00 |
+----------+---------+-----------+
+------------+-----------+-------------+
| first_name | last_name | enrollments |
+------------+-----------+-------------+
| Ana        | Lopez     |           3 |
| Chloe      | Martin    |           3 |
| Emma       | Novak     |           3 |
+------------+-----------+-------------+
+---------------------+-------------+---------+------------+
| title               | enrollments | ratings | avg_rating |
+---------------------+-------------+---------+------------+
| Fresh Pasta by Hand |           3 |       2 |        4.5 |
| Risotto Nights      |           2 |       2 |        4.5 |
| Sushi Basics        |           2 |       2 |        3.5 |
| Ramen from Scratch  |           2 |       2 |        4.0 |
| Sourdough Starter   |           2 |       1 |        5.0 |
| French Pastry       |           2 |       1 |        5.0 |
| Tacos and Salsas    |           2 |       2 |        3.0 |
```

```
+--------------------+------------+---------+------------+
+---------------------+-------+
| title               | price |
+---------------------+-------+
| French Pastry       | 75.00 |
| Fresh Pasta by Hand | 70.00 |
| Sushi Basics        | 70.00 |
+---------------------+-------+
+-------------+-------------+--------------------+--------------+
| enrolled_on | student     | class              | instructor   |
+-------------+-------------+--------------------+--------------+
| 2026-04-01  | Felix Grant | Risotto Nights     | Marco Rossi  |
| 2026-04-02  | Ben Okoro   | Ramen from Scratch | Aiko Tanaka  |
| 2026-04-05  | Felix Grant | Ramen from Scratch | Aiko Tanaka  |
| 2026-04-20  | Chloe Martin | French Pastry     | Lena Fischer |
| 2026-04-22  | Emma Novak  | French Pastry      | Lena Fischer |
+-------------+-------------+--------------------+--------------+
+------------+-----------+-----------------------+
| first_name | last_name | email                 |
+------------+-----------+-----------------------+
| Gina       | Russo     | gina.russo@example.com |
| Hugo       | Weber     | hugo.weber@example.com |
+------------+-----------+-----------------------+
+---------------+-------------+---------+
| name          | enrollments | revenue |
+---------------+-------------+---------+
| Marco Rossi   |           5 |  330.00 |
| Aiko Tanaka   |           4 |  260.00 |
| Lena Fischer  |           4 |  240.00 |
| Diego Morales |           2 |  100.00 |
| Nadia Haddad  |           0 |    0.00 |
+---------------+-------------+---------+
```

A3a at the prompt: `ERROR 1452 (23000): Cannot add or update a child row` (student 9) and `ERROR 3819 (HY000): Check constraint 'chk_enrollments_rating' is violated.` (rating 6). On MySQL 8.4 the same; on MariaDB the CHECK is `ERROR 4025 ... CONSTRAINT `chk_enrollments_rating` failed`.

TRAPS (AND WHAT THEY PRINT)

- **Part A3 skipped, B1 run anyway:** Fresh Pasta by Hand (65.00, not yet raised) joins the March list: three rows instead of two. Score B1 as right if the learner's own A3b is missing (score the missing A3b in its row), not twice.
- **B8 with `SUM(c.price)` and class 8 not deleted:** Diego Morales shows 180.00 instead of 100.00 (the empty class's 80.00 is summed once from its NULL-padded row) and Marco Rossi 305.00 (prices not raised). With A3 done, `SUM(c.price)` happens to print the reference numbers: accept it as Proficient, and award Exemplary only for a sum that would stay right with an empty class.
- **B8 with inner joins:** four instructors; Nadia Haddad (no classes) is missing. Developing on the joins row.

- **B4 with `AVG(COALESCE(rating, 0))`:** blanks count as zero stars: Fresh Pasta by Hand 3.0 instead of 4.5, Sourdough Starter and French Pastry 2.5 instead of 5.0. Developing on the aggregates row.
- **B7 with an inner join and IS NULL:** `Empty set`. An inner join has no unmatched rows.
- **B5 with the average typed in** (`WHERE price > 61.43`): the same three classes today (the average after Part A is 61.43), wrong after the next price change. Developing on the joins/subqueries row, and note it under Script quality.
- **B3 or B6 with ambiguous columns:** error 1052 stops the file at that query; the rest still runs if the learner turned it off. Script quality takes it.

WHAT AN HONEST RESULT SOUNDS LIKE

- A strong file reads like the reference: every answer under its TODO, a comment at A1's rules ("RESTRICT: a booking is a record of payment"), B8 with Nadia at 0.00.
- Follow-up questions for the feedback or a short conversation: "Why can a student enroll in a class only once in your table?" · "What would B8 print if a class had no enrollments?" · "Which of your queries would break if a price changed tomorrow?"
- A file with Part A complete and five of eight Part B answers right usually lands at 14 or 15: met. Part B answers that are wrong only because Part A was skipped are not penalized twice.

PART 1 ASSESSMENT: ACCOMMODATIONS, LATE WORK AND COMPLETION

- **Extra time:** up to 30 minutes in the room, agreed before Session 7; or the take-home variant (the same file, 48 hours, open book).
- **Missed Session 7:** the take-home variant within one week, plus a 10-minute conversation about two of the answers.
- **Academic honesty:** discussion after hand-in is encouraged; shared files are not. You may ask any learner to explain two of their queries in five minutes; an answer they cannot explain is re-scored after a short redo.
- **Part 1 on its own:** a learner completes Part 1 with the assessment met (or met after revision) and the seven sessions attended or made up. Checkpoint 1 (Session 4) is formative and not counted in the result; a "not yet" there predicts trouble in the aggregates row: visit those learners first during the assessment's Part B.
- **Part 2 follows:** the Part 1 result is recorded beside the final project (Sessions 12 to 14); a "not yet" on Part 1 does not block Part 2, but the learner redoes the weakest row's tasks before Session 9.

FINAL PROJECT: THE MAPLE STREET TOOL LIBRARY

Lesson 8.2 (`lesson_final_project.html`, 240 minutes for a solo learner). Brief handed out at the end of Session 12; Milestones 1 to 3 in the Session 13 studio; Milestone 4 at home; Milestones 5 to 7 and the showcase in Session 14; Milestone 8 and the README at home; due **one week after Session 14**.

THE BRIEF (ROSA'S WORDS, FROM THE LESSON)

Rosa Delgado coordinates the Maple Street Tool Library, a volunteer-run project that lends tools to its members. Her spreadsheet is falling apart: two people borrowed the same ladder on the same day, nobody knows who has the drill, and a member who never took the chainsaw course went home with the chainsaw. What the library needs:

1. "We have **members**: name, email (one account per email), the date they joined, and whether they are active or suspended."
2. "We own about a dozen **tools**. Each has a name, one **category** (hand tools, power tools, garden...), a condition (good, fair, or out for repair), what it would cost to replace, and the date we got it."
3. "Some tools need a **safety course** first: power tools need Power Tools Basics, the chainsaw needs Chainsaw Safety. We need to know which members finished which course, and when."
4. "A **loan** is one tool to one member: the day it went out, the day it is due (normally a week later) and the day it came back."
5. "Rules: a tool can only be out with one person at a time; nothing goes out while it is being repaired; suspended members cannot borrow; nobody may have more than three tools at once; and no course, no chainsaw."
6. "Every Saturday I need: what is on the shelf, what is overdue and whose email to write to, which tools are popular, and how busy each month was."
7. "The volunteers at the desk check tools out and in through our little web form. Our treasurer only reads reports. Nobody should be able to delete the loan history."
8. "If the laptop dies, I do not want to start over."

What learners hand in (lesson):

- `01_schema.sql`: creates the database and every table, and can be run again from scratch
- `02_data.sql`: realistic test data that exercises every rule
- `03_queries.sql`: Rosa's Saturday questions, one commented query each
- `04_logic.sql`: a view and the check-out and return procedures
- `05_security.sql`: roles and accounts
- A tested backup file and `README.md`: a one-page write-up of your design, how to rebuild it and what you would do next

OWN BRIEFS

- The lesson's FAQ allows another business if it has at least five tables, a many-to-many relationship, a rule that spans rows and two kinds of users. A ready-made alternative is on the page: a small physiotherapy clinic (patients, therapists, treatment types, appointments; a therapist cannot be booked twice at the same start time; cancelled appointments stay in the history; the front desk books, therapists only read). All data fictional.

- Approve own briefs in writing in Session 13. The rubric applies unchanged; the reference numbers below do not, so the learner's README must state their own expected results (the hand checks) and you check those.

WHERE THE 240 MINUTES GO

#	MILESTONE (LESSON)	USES LESSONS	LESSON ESTIMATE	WHERE IT HAPPENS IN THIS KIT
1	Design: entities, relationships, rules, diagram	4.1, 6.3	30 min	Session 13 studio (30 min; check-in per learner)
2	Build the schema and test the constraints	2.1, 2.2, 4.2	40 min	Session 13 studio (40 min; mini-demo of the test pattern first)
3	Populate with test data	2.3	25 min	Session 13 studio (25 min), finished at home
4	Answer the Saturday questions	3.3, 4.3, 5.1, 5.2	45 min	Home, between Sessions 13 and 14 (+ `edge_cases_starter.sql`)
5	Business logic: view and procedures	6.2, 7.1, 7.2	40 min	Session 14 studio (40 min; `add_item_rules_starter.sql` for stuck learners)
6	Security: roles and accounts	7.4	20 min	Session 14 studio (12 min)
7	Back up and test the restore	8.1	20 min	Session 14 studio (13 min)
8	Self-review and write-up	all	20 min	Home, within the week after Session 14

THE LESSON'S PROJECT COMPLETION CHECKLIST (WHAT "DONE" MEANS)

- A design with every requirement mapped to where it is enforced or answered
- `01_schema.sql` runs twice without errors and each constraint has a test that it rejects bad data
- `02_data.sql` covers the edge cases: suspended, never borrowed, in repair, overdue, late return, ties
- All six Saturday queries run against a fixed `@today` and at least two are checked by hand
- `checkout_tool` rejects each of the five rule violations with its own message and leaves no open transaction
- The desk and treasurer accounts work through roles, and neither is `root` or can delete
- A backup restored into a test database with matching counts, recorded in `README.md`

THE RUBRIC (LESSON 8.2, REPRODUCED EXACTLY)

Score yourself (or ask a classmate to): each area earns 0 (missing), half (meets), or full points (exceeds).

AREA	POINTS	MEETS (HALF THE POINTS)	EXCEEDS (FULL POINTS)
Design	15	Diagram and notes cover all five entities and their relationships	Every rule from the brief is mapped to where it is enforced, with reasons
Schema and constraints	20	Script runs twice without errors; keys and FKs correct	CHECKs and FK rules proven with failing test statements
Test data	10	Every table has rows; joins return results	Data deliberately covers edge cases: overdue, never borrowed, suspended, out for repair, ties
Queries	20	All six Saturday questions answered correctly	Readable CTEs, a window function, a fixed report date, results checked by hand
Business logic	15	Check-out procedure enforces most rules	All five rules, in a transaction, with clear error messages and tests for each
Security	10	App and reporting accounts that are not `root`	Roles; desk account can only change data through procedures
Backup and write-up	10	A dump file exists	Restore tested and logged; README explains how to rebuild everything

80 or more out of 100 is a solid, portfolio-ready project. The reference solution is written to score full points; yours can get there with a different design.

THE RESULT

- **Per area:** 0 (missing), half (meets) or full (exceeds), as the lesson says. Half of 15 is 7.5: record halves as they are; the total may end in .5.
- **Portfolio-ready (the lesson's bar):** 80 or more out of 100.
- **Completed (this kit's recommendation; adjust it to your program):** every area at least "meets", which is 50 or more with no area at 0. A project that reaches "meets" everywhere is a working, secured, backed-up database built from scripts.
- **Not yet:** any area at 0, or scripts that do not rebuild the database (score the rest only after a fix). Return written feedback (one strength per area and the single change that would gain the most points) and allow one resubmission within two weeks; re-score the changed areas only.
- **Rebuild first:** before reading anything else, run the learner's scripts in order on a clean server (the grading run below). If the rebuild needs a manual fix, ask for one within 48 hours, then score.

SCORING GUIDANCE BY AREA

AREA	MEETS (HALF)	EXCEEDS (FULL): LOOK FOR	QUICK TEST
Design (15)	A diagram or table list with the five entities (members, tools, categories, safety courses, loans) and their relationships; the junction table for courses.	All eight requirements mapped to where they are enforced or answered, with a reason each (the reference mapping is in the answer notes). Rules across rows placed in the procedure, not in CHECKs; derived data (overdue, available) not stored.	Pick requirement 5 and 7: can the design notes say where each part lives?
Schema and constraints (20)	`01_schema.sql` runs twice in a row without an error; every table has a primary key; foreign keys correct (types match, parents first).	Named constraints, at least two CHECKs, a deliberate ON DELETE choice per foreign key, and a failing test statement for each constraint (in a rolled-back transaction).	SOURCE it twice; run their tests: every test must be refused.
Test data (10)	Every table has rows; joins return results.	Planted edge cases a broken query would hide: overdue, never borrowed, suspended, out for repair, never lent, a tie, a late return.	Ask for the id of each edge case; check two in the data.
Queries (20)	All six Saturday questions answered correctly for their data (for the reference data: the outputs below).	CTEs that read well, a window function (RANK for ties, SUM OVER for the running total), one `@today` set once, and hand checks written down.	Change `@today` and rerun query 2: the reference gives 2 rows on June 15, 4 on June 20 and 1 on June 9 (answer notes).
Business logic (15)	A check-out procedure that enforces most of the five rules.	All five rules with their own messages, inside a transaction with a row lock and an EXIT HANDLER that rolls back and re-raises, `return_tool`, `v_open_loans`, and a failing call for every rule.	Run their failing calls; then call a success and check `v_open_loans`.
Security (10)	An application account and a reporting account that are not root.	Roles; SET DEFAULT ROLE; the desk account changes loans ONLY through the procedures (no INSERT/UPDATE/DELETE on loans); no DELETE anywhere.	SHOW GRANTS ... USING; log in as the desk account and try `DELETE FROM loans WHERE id = 1;` (must be 1142).

AREA	MEETS (HALF)	EXCEEDS (FULL): LOOK FOR	QUICK TEST
Backup and write-up (10)	A dump file exists.	The restore was tested (a test database, matching counts, the date) and logged in README.md; the README explains how to rebuild everything, including 05_security.sql (accounts are not in a dump).	Restore their dump into a test database yourself and compare one count.

THE GRADING RUN (ABOUT 20 MINUTES PER PROJECT)

1. Unzip into a fresh folder; `cd` into it; start the client THERE (`mysql -u root -p`): `02_data.sql` sources `01_schema.sql` by a relative path, which resolves from the client's folder.
2. `DROP DATABASE IF EXISTS tool_library;` then `SOURCE 02_data.sql` (it runs 01_schema.sql), `SOURCE 01_schema.sql` once more on its own (the rerun test), `SOURCE 02_data.sql` again, then `03_queries.sql` , `04_logic.sql` , `05_security.sql` (after replacing the placeholder passwords with throwaway ones). Note every ERROR that is not a deliberate test.
3. Compare the six query outputs with the reference outputs below (reference data) or with the learner's own hand checks.
4. Run the learner's failing calls; log in as their desk account in a second terminal and try one forbidden statement.
5. Restore their dump into `tool_library_check` and compare the loans count; then drop it, and drop their accounts and roles.
6. Read the README last, with the rubric beside it.

SCORING SHEET

Learner: _____ · Brief: reference / clinic / own · Date received: _____ · On time: yes / no · Rebuild clean: yes / after fix

AREA	POINTS	0 / HALF / FULL	SCORE	EVIDENCE (FILE AND LINE, OR WHAT YOU RAN)
Design	15			
Schema and constraints	20			
Test data	10			
Queries	20			
Business logic	15			

AREA	POINTS	0 / HALF / FULL	SCORE	EVIDENCE (FILE AND LINE, OR WHAT YOU RAN)
Security	10			
Backup and write-up	10			
Total	100			Result: portfolio-ready / completed / not yet

SUBMISSION AND ACADEMIC HONESTY

- **Deadline:** one week after Session 14. A zipped `tool-library` folder (or a repository link) with `01_schema.sql` to `05_security.sql`, the backup file of the test data, `README.md` with the restore date and counts, and the self-scored rubric (the lesson's "Before the Next Lesson" asks for the score and one improvement in the README).
- **Passwords:** placeholders only (`Change-Me-...`) in every script, as the lesson says. Never a `.mylogin.cnf`, never a backup of real data (the project's data is fictional).
- **The reference solution is on the lesson page**, hidden behind each milestone. Comparing after trying is expected (the lesson recommends it). A submission that IS the reference with no reasons scores "meets" at most on Design and cannot reach "exceeds" on Queries (no hand checks) or Business logic (no own tests). The README must say which parts were borrowed from the reference and why.
- **AI assistants and code from the web** are allowed on the terms a working developer accepts: the learner is responsible for every line and says in the README what help was used. **The explain-two-statements check:** pick any two statements and ask the learner to explain, in five minutes, what they do and why they are right; anything the learner cannot explain is revised before it is scored.
- **Shared ideas, not shared files:** learners may compare designs and error messages; scripts and the README are their own.
- Do not share these notes with learners; feedback refers to the rubric and their own files.

LATE WORK AND ALTERNATIVES

- **Extension:** one extra week on request before the deadline, no reason needed; later requests at your discretion.
- **Late without an extension:** scored as usual; record the date. The rubric measures the work.
- **Missed Session 13 or 14:** the milestones are on the lesson page with references; book a 15-minute check-in instead of the missed studio check-ins. Missed showcase: a three-minute screen recording (one design decision, one checked answer, one refused call).
- **Narrower scope agreed in advance** (for example for a learner who joined at Part 2 without Part 1): the reference design typed from the page, then the learner's own constraint tests, own Saturday queries, own failing calls and own security script. Design is then scored on the written reasons only.

- **Accessibility:** the showcase can be a written card or a recording; the README can be dictated.

RECORDING COURSE RESULTS

The course has two formative checkpoints and two summative assessments. Checkpoints are scored with their own five-row rubrics (Session 4 and Session 11 answer keys) and are not weighted into the assessments.

LEARNER	CHECKPOINT 1 (S4)	PART 1 ASSESSMENT (S7)	CHECKPOINT 2 (S11)	FINAL PROJECT /100	RESULT

- **Full program completion (kit recommendation):** the Part 1 assessment passed (its own rule, in the Part 1 section above) and the final project completed (every area at least "meets"). Checkpoints: Met, or redone once at home.
- **Part 1 only:** the Part 1 assessment is the course's final assessment; Checkpoint 1 is recorded beside it.
- **Part 2 only** (learners who already have the Part 1 skills): the final project alone, with Checkpoint 2 beside it.

ANSWER NOTES · FINAL PROJECT REFERENCE NUMBERS

Every output below comes from running Lesson 8.2's own reference solution on MySQL 9.7.2 in one client session, exactly in the page's order (`kit_generators/final/ref_run.py`; every output matches the lesson page). The account logins, the dump and restore, and the rebuild test were run separately by logging in as the accounts and calling mysqldump. Use them for learners who follow the reference data; learners with their own data are checked against their own hand checks.

MILESTONE 1: THE REFERENCE MAPPING

REQUIREMENT	WHERE IT LIVES IN THE REFERENCE
1 to 4, the facts	Columns, NOT NULL, UNIQUE (email, category and course names), ENUMs (status, tool_condition), foreign keys
5, the five rules	`checkout_tool`, in a transaction with a row lock (they look at other rows)
6, the Saturday questions	`03_queries.sql` and the view `v_open_loans`
7, desk, treasurer, no deleting	Roles `tl_desk` and `tl_reports`; no DELETE granted anywhere; the loans foreign keys refuse deleting a member or tool with history
8, the dead laptop	mysqldump with a tested restore; the numbered scripts kept in version control

Defensible alternatives: a lookup table for conditions or statuses, one `name` column for members, a `loan_status` column (only if it cannot disagree with `returned_on`: ask how), ON DELETE CASCADE on member_courses (the reference does it) or none. Wrong: storing anything twice, losing a fact (no `returned_on`), or a rule from the brief with no home.

MILESTONE 2: THE SCHEMA AND ITS TESTS

`01_schema.sql` (64 lines) ends with SHOW TABLES:

```sql
DROP DATABASE IF EXISTS tool_library;
CREATE DATABASE tool_library CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE tool_library;

CREATE TABLE members (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50)  NOT NULL,
    last_name  VARCHAR(50)  NOT NULL,
    email      VARCHAR(100) NOT NULL,
    joined_on  DATE         NOT NULL,
    status     ENUM('active', 'suspended') NOT NULL DEFAULT 'active',
    CONSTRAINT uq_members_email UNIQUE (email)
);

CREATE TABLE categories (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    CONSTRAINT uq_categories_name UNIQUE (name)
);

CREATE TABLE safety_courses (
    id   INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(80) NOT NULL,
    CONSTRAINT uq_courses_name UNIQUE (name)
);

CREATE TABLE tools (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    name              VARCHAR(100)  NOT NULL,
    category_id       INT           NOT NULL,
    required_course_id INT          NULL,
    tool_condition    ENUM('good', 'fair', 'repair') NOT NULL DEFAULT 'good',
    replacement_value DECIMAL(8, 2) NOT NULL,
    acquired_on       DATE          NOT NULL,
    CONSTRAINT fk_tools_category FOREIGN KEY (category_id) REFERENCES categories (id),
    CONSTRAINT fk_tools_course   FOREIGN KEY (required_course_id) REFERENCES safety_cou
    CONSTRAINT chk_tools_value   CHECK (replacement_value > 0)
);

CREATE TABLE member_courses (
    member_id    INT  NOT NULL,
    course_id    INT  NOT NULL,
    completed_on DATE NOT NULL,
    PRIMARY KEY (member_id, course_id),
    CONSTRAINT fk_mc_member FOREIGN KEY (member_id) REFERENCES members (id) ON DELETE C
    CONSTRAINT fk_mc_course FOREIGN KEY (course_id) REFERENCES safety_courses (id)
);
```

```
CREATE TABLE loans (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    tool_id     INT  NOT NULL,
    member_id   INT  NOT NULL,
    loaned_on   DATE NOT NULL,
    due_on      DATE NOT NULL,
    returned_on DATE NULL,
    CONSTRAINT fk_loans_tool      FOREIGN KEY (tool_id)   REFERENCES tools (id),
    CONSTRAINT fk_loans_member    FOREIGN KEY (member_id) REFERENCES members (id),
    CONSTRAINT chk_loans_due      CHECK (due_on >= loaned_on),
    CONSTRAINT chk_loans_returned CHECK (returned_on IS NULL OR returned_on >= loaned_o
    INDEX idx_loans_tool_open   (tool_id, returned_on),
    INDEX idx_loans_member_open (member_id, returned_on)
);

SHOW TABLES;
```

```
Database changed
+------------------------+
| Tables_in_tool_library |
+------------------------+
| categories             |
| loans                  |
| member_courses         |
| members                |
| safety_courses         |
| tools                  |
+------------------------+
```

The tests, inside one transaction:

```
START TRANSACTION;

INSERT INTO categories (name) VALUES ('Hand Tools');
INSERT INTO members (first_name, last_name, email, joined_on)
VALUES ('Test', 'Member', 'test@example.org', '2026-01-01');
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Test hammer', 1, 15.00, '2026-01-01');
```

```
INSERT INTO loans (tool_id, member_id, loaned_on, due_on)
VALUES (1, 1, '2026-06-10', '2026-06-03');
```

```
ERROR 3819 (HY000): Check constraint 'chk_loans_due' is violated.
```

```
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Free saw', 1, 0, '2026-01-01');
```

```
ERROR 3819 (HY000): Check constraint 'chk_tools_value' is violated.
```

```
INSERT INTO tools (name, category_id, replacement_value, acquired_on)
VALUES ('Mystery tool', 99, 10.00, '2026-01-01');
```

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (
```

```
INSERT INTO members (first_name, last_name, email, joined_on)
VALUES ('Second', 'Account', 'test@example.org', '2026-02-01');
```

```
ERROR 1062 (23000): Duplicate entry 'test@example.org' for key 'members.uq_members_ema
```

```
ROLLBACK;

SELECT COUNT(*) AS members_left FROM members;
```

```
+--------------+
| members_left |
+--------------+
|            0 |
+--------------+
```

Rerun test (measured): sourcing the schema twice in a row prints the same six tables twice and no ERROR. Foreign-key delete rules as information_schema reports them (Lesson 8.3's check 3 on tool_library): five `NO ACTION` (the script writes no rule: "refuse") and `fk_mc_member` CASCADE:

```
+----------------+-------------------+-----------------------+-------------+
| TABLE_NAME     | CONSTRAINT_NAME   | REFERENCED_TABLE_NAME | DELETE_RULE |
+----------------+-------------------+-----------------------+-------------+
| loans          | fk_loans_member   | members               | NO ACTION   |
| loans          | fk_loans_tool     | tools                 | NO ACTION   |
| member_courses | fk_mc_course      | safety_courses        | NO ACTION   |
| member_courses | fk_mc_member      | members               | CASCADE     |
| tools          | fk_tools_category | categories            | NO ACTION   |
| tools          | fk_tools_course   | safety_courses        | NO ACTION   |
+----------------+-------------------+-----------------------+-------------+
```

Lesson 8.3's other checks on the reference tool_library: 6 tables; no table without a primary key, no FLOAT or non-utf8mb4 column, no foreign-key column without its own index (all Empty set).

MILESTONE 3: THE DATA

Counts after `02_data.sql` (the same after a full rebuild from the files, measured):

```
+----------------+-----------+
| table_name     | row_count |
+----------------+-----------+
| members        |         8 |
| categories     |         5 |
| safety_courses |         2 |
| member_courses |         7 |
| tools          |        12 |
| loans          |        16 |
+----------------+-----------+
```

- Victor (8) suspended; Tessa (7) never borrowed; the tile cutter (12) in repair and never lent; the hammer (1) and the socket set (4) tied in Hand Tools with one loan each; loan 5 returned two days late (due March 14, back March 16); loans 13 to 16 open.

MILESTONE 4: THE SIX SATURDAY ANSWERS (`@TODAY = '2026-06-15'`)

```sql
SET @today = '2026-06-15';

-- 1. On the shelf now
SELECT t.id, t.name, c.name AS category
FROM tools AS t
JOIN categories AS c ON c.id = t.category_id
WHERE t.tool_condition <> 'repair'
  AND NOT EXISTS (SELECT 1 FROM loans AS l
                  WHERE l.tool_id = t.id AND l.returned_on IS NULL)
ORDER BY c.name, t.name;
```

```
+----+-----------------+-------------+
| id | name            | category    |
+----+-----------------+-------------+
|  9 | Pressure washer | Cleaning    |
|  6 | Chainsaw        | Garden      |
|  5 | Hedge trimmer   | Garden      |
| 11 | Wheelbarrow     | Garden      |
|  1 | Claw hammer     | Hand Tools  |
|  8 | Step ladder     | Ladders     |
|  3 | Circular saw    | Power Tools |
+----+-----------------+-------------+
```

```sql
-- 2. Overdue, worst first
SELECT l.id AS loan_id, t.name AS tool,
       CONCAT(m.first_name, ' ', m.last_name) AS member, m.email,
       l.due_on, DATEDIFF(@today, l.due_on) AS days_overdue
FROM loans AS l
JOIN tools   AS t ON t.id = l.tool_id
JOIN members AS m ON m.id = l.member_id
WHERE l.returned_on IS NULL
  AND l.due_on < @today
ORDER BY days_overdue DESC;
```

```
+---------+------------------+--------------+---------------------------+------------+--
| loan_id | tool             | member       | email                     | due_on     | d
+---------+------------------+--------------+---------------------------+------------+--
|      13 | Extension ladder | Amara Okoye  | amara.okoye@example.org   | 2026-06-08 |
|      14 | Cordless drill   | Rosa Delgado | rosa.delgado@example.org  | 2026-06-13 |
+---------+------------------+--------------+---------------------------+------------+--
```

```sql
-- 3. Most borrowed per category (RANK keeps ties)
WITH loan_counts AS (
    SELECT c.name AS category, t.name AS tool, COUNT(l.id) AS loans
    FROM tools AS t
    JOIN categories AS c ON c.id = t.category_id
    LEFT JOIN loans AS l ON l.tool_id = t.id
    GROUP BY c.name, t.id, t.name
),
ranked AS (
    SELECT category, tool, loans,
           RANK() OVER (PARTITION BY category ORDER BY loans DESC) AS rank_in_category
    FROM loan_counts
)
SELECT category, tool, loans
FROM ranked
WHERE rank_in_category = 1
ORDER BY category, tool;
```

```
+-------------+------------------+-------+
| category    | tool             | loans |
+-------------+------------------+-------+
| Cleaning    | Pressure washer  |     2 |
| Garden      | Chainsaw         |     2 |
| Hand Tools  | Claw hammer      |     1 |
| Hand Tools  | Socket set       |     1 |
| Ladders     | Extension ladder |     2 |
| Power Tools | Cordless drill   |     4 |
+-------------+------------------+-------+
```

```sql
-- 4. Loans per month with a running total
WITH monthly AS (
    SELECT DATE_FORMAT(loaned_on, '%Y-%m') AS month, COUNT(*) AS loans
    FROM loans
    GROUP BY DATE_FORMAT(loaned_on, '%Y-%m')
)
SELECT month, loans,
       SUM(loans) OVER (ORDER BY month) AS running_total
FROM monthly
ORDER BY month;
```

```
+---------+-------+---------------+
| month   | loans | running_total |
+---------+-------+---------------+
| 2026-01 |     2 |             2 |
| 2026-02 |     2 |             4 |
| 2026-03 |     2 |             6 |
| 2026-04 |     3 |             9 |
| 2026-05 |     3 |            12 |
| 2026-06 |     4 |            16 |
+---------+-------+---------------+
```

```sql
-- 5. Never borrowed
SELECT CONCAT(m.first_name, ' ', m.last_name) AS member, m.status
FROM members AS m
LEFT JOIN loans AS l ON l.member_id = m.id
WHERE l.id IS NULL
ORDER BY member;
```

```
+------------------+-----------+
| member           | status    |
+------------------+-----------+
| Tessa Morgan     | active    |
| Victor Lindqvist | suspended |
+------------------+-----------+
```

```sql
-- 6. Active members cleared for the chainsaw
SELECT CONCAT(m.first_name, ' ', m.last_name) AS member, mc.completed_on
FROM tools AS t
JOIN member_courses AS mc ON mc.course_id = t.required_course_id
JOIN members AS m         ON m.id = mc.member_id
WHERE t.name = 'Chainsaw'
  AND m.status = 'active'
ORDER BY member;
```

```
+--------------+--------------+
| member       | completed_on |
+--------------+--------------+
| Liam Foster  | 2026-03-14   |
| Rosa Delgado | 2025-10-04   |
+--------------+--------------+
```

The lesson's hand checks: tools 2, 4, 7 and 10 are out and 12 is in repair, so 7 of 12 are on the shelf; on June 15 the ladder due June 8 is 7 days late and the drill due June 13 is 2. The query-2 answer moves with the report date (measured):

`@today = '2026-06-20'`:

```
+---------+------------------+--------------+---------------------------+------------+--
| loan_id | tool             | member       | email                     | due_on     | d
+---------+------------------+--------------+---------------------------+------------+--
|      13 | Extension ladder | Amara Okoye  | amara.okoye@example.org   | 2026-06-08 |
|      14 | Cordless drill   | Rosa Delgado | rosa.delgado@example.org  | 2026-06-13 |
|      15 | Socket set       | Kevin Brooks | kevin.brooks@example.org  | 2026-06-17 |
|      16 | Carpet cleaner   | Hannah Weiss | hannah.weiss@example.org  | 2026-06-19 |
+---------+------------------+--------------+---------------------------+------------+--
```

`@today = '2026-06-09'`:

```
+---------+------------------+-------------+--------------------------+------------+---
| loan_id | tool             | member      | email                    | due_on     | day
+---------+------------------+-------------+--------------------------+------------+---
|      13 | Extension ladder | Amara Okoye | amara.okoye@example.org | 2026-06-08 |
+---------+------------------+-------------+--------------------------+------------+---
```

- **Grading a different but correct query:** compare results, not text. A LEFT JOIN ... IS NULL instead of NOT EXISTS in query 1 is fine; ROW_NUMBER instead of RANK in query 3 drops the Hand Tools tie (5 rows instead of 6: wrong); a condition on the loans side moved into the WHERE of a LEFT JOIN (the lesson's Mistake 5: rows vanish), an inner JOIN in query 5 (Empty set), and a monthly query grouped by MONTH() alone (it would merge the same month of two years; the reference data has one year, so ask about it) are the usual wrong answers.
- Queries that use `CURDATE()` instead of `@today` cannot be checked against these numbers and miss "a fixed report date" (exceeds).

MILESTONE 5: THE LOGIC, ANSWERING

`04_logic.sql` (77 lines: the view, `checkout_tool`, `return_tool`) runs without output. The calls:

```
CALL checkout_tool(8, 1, '2026-06-15');   -- Victor is suspended
CALL checkout_tool(1, 12, '2026-06-15');  -- tile cutter in repair
CALL checkout_tool(6, 7, '2026-06-15');   -- ladder is out with Amara
CALL checkout_tool(7, 6, '2026-06-15');   -- Tessa has no chainsaw course
```

```
ERROR 1644 (45000): Member is suspended
ERROR 1644 (45000): Tool is out for repair
ERROR 1644 (45000): Tool is already on loan
ERROR 1644 (45000): Safety course required for this tool
```

```
CALL checkout_tool(2, 1, '2026-06-15');
CALL checkout_tool(2, 3, '2026-06-15');
CALL checkout_tool(2, 8, '2026-06-15');   -- a fourth tool for Kevin
```

```
+---------+------------+
| loan_id | due_on     |
+---------+------------+
|      17 | 2026-06-22 |
+---------+------------+
+---------+------------+
| loan_id | due_on     |
+---------+------------+
|      18 | 2026-06-22 |
+---------+------------+
ERROR 1644 (45000): Loan limit reached (3 open loans)
```

```
CALL return_tool(1, '2026-06-16');
CALL return_tool(3, '2026-06-16');
CALL return_tool(7, '2026-06-16');

SELECT * FROM v_open_loans ORDER BY due_on, loan_id;

CALL return_tool(7, '2026-06-16');        -- the ladder is already back
```

```
+---------+---------+----------------+--------------+------------+------------+
| loan_id | tool_id | tool           | member       | loaned_on  | due_on     |
+---------+---------+----------------+--------------+------------+------------+
|      14 |       2 | Cordless drill | Rosa Delgado | 2026-06-06 | 2026-06-13 |
|      15 |       4 | Socket set     | Kevin Brooks | 2026-06-10 | 2026-06-17 |
|      16 |      10 | Carpet cleaner | Hannah Weiss | 2026-06-12 | 2026-06-19 |
+---------+---------+----------------+--------------+------------+------------+
ERROR 1644 (45000): Tool is not on loan
```

After Milestone 5 the reference has **18 loans** (16 + Kevin's 17 and 18), 3 open (14, 15, 16). A learner's "exceeds" needs each of the five rules refused with its own message and no open transaction afterwards (the EXIT HANDLER with ROLLBACK and RESIGNAL). A handler that rolls back without RESIGNAL makes a refused call look successful to the caller (no error reaches the web form; Lesson 7.2): score "meets" and say why in the feedback.

MILESTONE 6: SECURITY, MEASURED BY LOGGING IN

`05_security.sql` ends with:

```
CREATE ROLE IF NOT EXISTS 'tl_desk', 'tl_reports';

GRANT SELECT ON tool_library.* TO 'tl_reports';

GRANT SELECT ON tool_library.* TO 'tl_desk';
GRANT INSERT, UPDATE ON tool_library.members TO 'tl_desk';
GRANT EXECUTE ON PROCEDURE tool_library.checkout_tool TO 'tl_desk';
GRANT EXECUTE ON PROCEDURE tool_library.return_tool TO 'tl_desk';

CREATE USER IF NOT EXISTS 'tl_desk_app'@'localhost' IDENTIFIED BY 'Change-Me-Desk-2026'
GRANT 'tl_desk' TO 'tl_desk_app'@'localhost';
SET DEFAULT ROLE 'tl_desk' TO 'tl_desk_app'@'localhost';

CREATE USER IF NOT EXISTS 'tl_treasurer'@'localhost' IDENTIFIED BY 'Change-Me-Reports-2
GRANT 'tl_reports' TO 'tl_treasurer'@'localhost';
SET DEFAULT ROLE 'tl_reports' TO 'tl_treasurer'@'localhost';

SHOW GRANTS FOR 'tl_desk_app'@'localhost' USING 'tl_desk';
```

```
+-----------------------------------------------------------------------------------------
| Grants for tl_desk_app@localhost
+-----------------------------------------------------------------------------------------
| GRANT USAGE ON *.* TO `tl_desk_app`@`localhost`
| GRANT SELECT ON `tool_library`.* TO `tl_desk_app`@`localhost`
| GRANT INSERT, UPDATE ON `tool_library`.`members` TO `tl_desk_app`@`localhost`
| GRANT EXECUTE ON PROCEDURE `tool_library`.`checkout_tool` TO `tl_desk_app`@`localhost
| GRANT EXECUTE ON PROCEDURE `tool_library`.`return_tool` TO `tl_desk_app`@`localhost`
| GRANT `tl_desk`@`%` TO `tl_desk_app`@`localhost`
+-----------------------------------------------------------------------------------------
```

Logged in as `tl_desk_app` (password from the script), each statement separately:

```
SELECT CURRENT_USER() AS logged_in_as, CURRENT_ROLE() AS active_role;
CALL checkout_tool(4, 1, '2026-06-17');
DELETE FROM loans WHERE id = 1;
INSERT INTO loans (tool_id, member_id, loaned_on, due_on) VALUES (3, 4, '2026-06-17', '
UPDATE members SET last_name = 'Foster' WHERE id = 4;
```

```
+-----------------------+---------------+
| logged_in_as          | active_role   |
+-----------------------+---------------+
| tl_desk_app@localhost | `tl_desk`@`%` |
+-----------------------+---------------+
+---------+------------+
| loan_id | due_on     |
+---------+------------+
|      19 | 2026-06-24 |
+---------+------------+
ERROR 1142 (42000): DELETE command denied to user 'tl_desk_app'@'localhost' for table
ERROR 1142 (42000): INSERT command denied to user 'tl_desk_app'@'localhost' for table
(no output: UPDATE on members is allowed)
```

Logged in as `tl_treasurer`:

```
SELECT COUNT(*) AS loans FROM loans;
UPDATE tools SET replacement_value = 1 WHERE id = 1;
```

```
+-------+
| loans |
+-------+
|    19 |
+-------+
ERROR 1142 (42000): UPDATE command denied to user 'tl_treasurer'@'localhost' for table
```

The desk account creates loan 19 through the procedure (definer's rights) while its own INSERT and DELETE on loans are refused: that is "desk account can only change data through procedures" (exceeds). The treasurer counts 19 loans (the desk's demo loan included) and cannot change anything.

MILESTONE 7: THE BACKUP AND THE RESTORE TEST

The lesson's mysqldump command on the reference after Milestone 5 (the desk's demo loan 19 removed again): exit code 0, 338 lines, 15,284 bytes, no warnings. Inside the file: CREATE TABLE for all six tables, the view, both procedures with `DEFINER=\`root\`@\`localhost\`` ; no `CREATE USER` and no `GRANT` (accounts are not in a single-database dump). Restored into `tool_library_test` as root (exit code 0):

```
+----------------+------------+
| loans_original | loans_copy |
+----------------+------------+
|             18 |         18 |
+----------------+------------+
+---------------+--------------+----------------+
| ROUTINE_NAME  | ROUTINE_TYPE | DEFINER        |
+---------------+--------------+----------------+
| checkout_tool | PROCEDURE    | root@localhost |
| return_tool   | PROCEDURE    | root@localhost |
+---------------+--------------+----------------+
+--------------------+
| open_loans_in_copy |
+--------------------+
|                  3 |
+--------------------+
```

Both counts 18; both procedures; the copy's view lists the 3 open loans. A learner restoring as a non-root account may hit a definer error: the lesson says to restore as root.

MILESTONE 8 AND THE README

- Rebuild from the files in order (02_data.sql, which runs 01_schema.sql; then 03, 04, 05) on a clean server: the reference gives the counts above and no ERROR except the deliberate tests.
- The README skeleton is on the lesson page (Design, Rebuild from scratch, Backups with the last restore date and "counts matched", Next steps). "Exceeds" needs the restore date and counts and the security script in the rebuild steps.

THE LESSON'S EXERCISES (HOMEWORK AFTER SESSION 14), REFERENCE OUTPUTS

They run on tool_library as Milestone 5 leaves it (18 loans).

```
SELECT l.id AS loan_id, t.name AS tool,
       CONCAT(m.first_name, ' ', m.last_name) AS member,
       l.due_on, l.returned_on,
       DATEDIFF(l.returned_on, l.due_on) AS days_late
FROM loans AS l
JOIN tools   AS t ON t.id = l.tool_id
JOIN members AS m ON m.id = l.member_id
WHERE l.returned_on > l.due_on
ORDER BY days_late DESC, loan_id;
```

```
+---------+------------------+-------------+------------+-------------+-----------+
| loan_id | tool             | member      | due_on     | returned_on | days_late |
+---------+------------------+-------------+------------+-------------+-----------+
|      13 | Extension ladder | Amara Okoye | 2026-06-08 | 2026-06-16  |         8 |
|       5 | Pressure washer  | Amara Okoye | 2026-03-14 | 2026-03-16  |         2 |
+---------+------------------+-------------+------------+-------------+-----------+
```

```
SELECT CONCAT(m.first_name, ' ', m.last_name) AS member,
       COUNT(l.id) AS total_loans,
       COUNT(CASE WHEN l.id IS NOT NULL AND l.returned_on IS NULL THEN 1 END) AS open_l
       MAX(l.loaned_on) AS latest_loan
FROM members AS m
LEFT JOIN loans AS l ON l.member_id = m.id
GROUP BY m.id, m.first_name, m.last_name
ORDER BY total_loans DESC, member;
```

```
+-----------------+-------------+------------+-------------+
| member          | total_loans | open_loans | latest_loan |
+-----------------+-------------+------------+-------------+
| Kevin Brooks    |           5 |          1 | 2026-06-15  |
| Rosa Delgado    |           4 |          1 | 2026-06-06  |
| Hannah Weiss    |           3 |          1 | 2026-06-12  |
| Amara Okoye     |           2 |          0 | 2026-06-01  |
| Liam Foster     |           2 |          0 | 2026-04-18  |
| Omar Haddad     |           2 |          0 | 2026-05-23  |
| Tessa Morgan    |           0 |          0 | NULL        |
| Victor Lindqvist |           0 |          0 | NULL        |
+-----------------+-------------+------------+-------------+
```

```
SELECT IF(GROUPING(c.name), 'All categories', c.name) AS category,
       COUNT(*) AS tools_out,
       SUM(t.replacement_value) AS value_out
FROM v_open_loans AS o
JOIN tools      AS t ON t.id = o.tool_id
JOIN categories AS c ON c.id = t.category_id
GROUP BY c.name WITH ROLLUP;
```

```
+----------------+-----------+-----------+
| category       | tools_out | value_out |
+----------------+-----------+-----------+
| Cleaning       |         1 |    200.00 |
| Hand Tools     |         1 |     60.00 |
| Power Tools    |         1 |    120.00 |
| All categories |         3 |    380.00 |
+----------------+-----------+-----------+
```

```
ALTER TABLE loans
    ADD COLUMN open_tool_id INT
        GENERATED ALWAYS AS (IF(returned_on IS NULL, tool_id, NULL)) STORED,
    ADD CONSTRAINT uq_loans_one_open_per_tool UNIQUE (open_tool_id);
```

```
INSERT INTO loans (tool_id, member_id, loaned_on, due_on)
VALUES (2, 4, '2026-06-16', '2026-06-23');   -- the drill is out with Rosa
```

```
ERROR 1062 (23000): Duplicate entry '2' for key 'loans.uq_loans_one_open_per_tool'
```

The lesson's own Quick Quiz answers: B-A-C-A-C (also in the Session 14 Answer Key).

ERRORS LEARNERS WILL SEND YOU, AND WHAT THEY MEAN

ERROR	CAUSE IN THIS PROJECT
1064 near 'condition'	A reserved word as a column name (also order, group, rank, key)
3780 ... are incompatible	A foreign key's type or signedness differs from its target
1824 Failed to open the referenced table	A child table created before its parent
3814 ... disallowed function: curdate	Today's date in a CHECK
3819 Check constraint ... is violated	A real data error (dates swapped), or a test doing its job

ERROR	CAUSE IN THIS PROJECT
1452 Cannot add or update a child row	A row pointing at an id that does not exist (yet)
1644 (45000) + a message	The procedure's own SIGNAL: a rule refused the call (expected in tests)
1142 ... command denied	The account lacks the privilege (expected for the desk's DELETE), or the role was never activated
Failed to open file '01_schema.sql', error: 2	The client was started outside the project folder (nested SOURCE paths)

© 2026 Ray de la Paz · MySQL Foundations